CAHIER DES CHARGES

Shooter like

Gaming Campus - © Active Learning by Quest Education Group
Document Interne 2024-2025

INTRODUCTION

Nom du projet : Shooter like – Shoot'em up
Nº du Projet : 04
Nom d'UE : Projets appliqués
Nº d'UE : 03
Date : 02/12/2024 – 20/12/2024
Classe : G-TECH 1

Modalités

Projet en groupe de 2 à 3 maximum.

Contexte et Description du Projet

L'objectif de ce projet est de s'intéresser à un style de jeu qui existe depuis les débuts du jeu vidéo. En effet, le but est de réaliser **un shoot'em ou shooter like**. Afin de vous faire une idée sur ce qu'est un shooter, vous pouvez vous inspirez de titres populaires.

Figure 2 Asteroids sur Atari en 1979

Figure 1 Vampire Survivors en 2022

L'objectif de ce projet est donc de réaliser un jeu vidéo avec un **rendu graphique**. Le jeu sera bien entendu, développé entièrement en C++ et vous aurez droit seulement à l'utilisation de la **librairie SFML** en plus de **la librairie standard.** Vous

pourrez utiliser le dernier standard C++23 si vous le souhaitez. On autorise ImGUI si nécessaire mais ne perdez pas de temps avec cette dernière librairie.

Vous aurez la liberté totale sur la partie design graphique mais n'hésitez pas à créer votre propre univers. On a déjà eu des shooters où l'on était un survivant qui faisait face à des Zombies, et leur tirait dessus avec des pistolets et des fusils à pompe, donc amusez-vous !

Pour résumer le principe d'un shooter : le joueur contrôle un objet graphique (vaisseau, personnage, autre…) et il est attaqué par différents types d'ennemis qui peuvent lui faire des dégâts au corps à corps ou à distance ou même des dégâts de zone.

Les possibilités d'environnements d'un shooter sont grandes, vous pourrez décider que le joueur puisse se mouvoir sur une carte finie ou infinie ou même faire en sorte que le joueur se déplace en bottom-top à la space invaders.

Gaming Campus - © Active Learning by Quest Education Group
Document Interne 2024-2025

Specs

Vous devez implémenter dans votre application :

- Gameplay
  - Menu d'écran d'accueil
    - Quand le joueur lance le jeu, un écran d'accueil doit s'afficher et il est possible de sélectionner un élément parmi les suivants :
      - Play
        - Permet de lancer le jeu
      - High-Score
        - Permet de visualiser les 10 meilleurs scores
      - Exit
        - Permet de quitter **proprement** le jeu
  - Navigation joueur
    - Le joueur utilisera les touches directionnelles pour se déplacer.
    - La souris pourra être utilisée pour orienter le projectile et pour effectuer des rotations du player en fonction du shooter proposé.
    - La barre d'espace ou le clic souris sera utilisé pour attaquer.
  - ATH joueur
    - Le joueur doit disposer d'une barre de vie ou cœurs ou tout autre élément visuel capable de montrer l'état de sa santé avant de mourir.
    - Le joueur doit être en mesure de voir son score affiché tout au long de la partie.
  - Mort du joueur
    - Lorsque le joueur perd, un game over doit apparaitre.

        - Si le score rentre dans le top ten alors, le joueur est invité à rentrer un trigramme à l'ancienne pour sauvegarder son score.

    - Bonus/Malus system
        - Mettre en place un mécanisme où le player peut avoir des power-up ou power-down qui affecteront le joueur (déplacement, vie, dégâts, etc…)

- Performances
    - Le jeu doit être synchronisé sur 60fps et on doit pouvoir afficher le framerate via une touche clavier.
        - Celui-ci ne doit pas chuter lorsque des nouveaux ennemis poppent dans la zone de combat.

- Collisions
    - Un système de collision AABB devra être mis en place pour gérer la collision des objets.

- Outils
    - Les étudiants auront la possibilité d'utiliser la librairie standard C++17/C++ 20 ou C++23
    - Ils devront versionner leur code en utilisant git afin d'avoir une phase de développement saine.
        - Ils devront créer des branches avec des merge request.
        - Ils devront avoir un git léger qui contient le strict minimum requis pour cloner et recompiler.
    - Pour les assets, il sera possible d'utiliser des sprites récupérés sur Internet à condition que ceux-ci respectent les licences d'utilisation et de modifications. Bien entendu, si les étudiants ont la fibre artistique ou ont des camarades qui produisent des assets, ceux-ci peuvent être utilisés.

- Afin que les membres de l’équipe communiquent facilement le long de la réalisation de leur projet, il est indispensable d’utiliser des outils propres au développement logiciel :
    - Traces UML
    - Traces des décisions
    - Etc… et cela pourra être présenté le jour de la soutenance

Pour les plus avancés uniquement (ou pour aller plus loin)

- **Multi-weapon :** mettre en place un mécanisme qui permettra au joueur d'avoir différentes armes avec différentes caractéristiques. Celles-ci, pourront être sélectionnables via un mécanisme suffisamment ergonomique.
- **Vie des ennemis :** vous pouvez ajouter des ennemis qui ne meurt pas en un seul coup mais qui disposent d'une résistance plus forte.
- **Comportements ennemis :** les ennemis peuvent avoir des comportements divers et variés.
- **Création de Boss :** lorsque le joueur atteint des paliers, il peut faire face à des boss.

Gaming Campus - © Active Learning by Quest Education Group
Document Interne 2024-2025

OBJECTIFS PÉDAGOGIQUES ET PROFESSIONNELS DU PROJET

A l'issue de ce projet, les étudiants seront capables de :

Savoirs

- **Modéliser** des problèmes architecturaux de plus haut niveau.
- **Identifier** les problématiques techniques usuelles de l'industrie.
- **Identifier et développer** les principales briques logicielles nécessaires au développement de jeux vidéo.
- **Comprendre** les notions de performances et de framerate.
- **Discriminer** les parties logicielles qui affectent les performances.
- **Expliquer** ce qu'est la notion de gameplay.
- **Faire le lien** entre le code et le plaisir en jeu.
- **Exploiter** du contenu fait par des artistes et les intégrer dans un jeu.

Savoir-être

- **Construire** une solution propre et pérenne pour développer en équipe.
- **Découper et paralléliser** des tâches en adéquation avec l'architecture.
- **S'organiser** pour avoir des MVP (Minimum Viable Product) le plus souvent possible.

Savoir-faire / Compétences

- **Programmer** en utilisant des concepts nécessaires dans l'industrie.
- **Utiliser** des ressources artistiques.
- **Consommer** des librairies tierces

RESSOURCES

1. **A installer/prérequis techniques** :
    - Visual Studio 2022
    - Git
    - Helix (p4merge)
    - Vcpkg
    - SFML

Gaming Campus - © Active Learning by Quest Education Group
Document Interne 2024-2025

INTERVENANTS

Nom : OUKRAT Rémi
Titre : Architecte logiciel
Email : remi.oukrat@gmail.com
LinkedIn : www.linkedin.com/in/remi-o

BIOGRAPHIE

Depuis mon plus jeune âge, je suis passionné par les jeux vidéo. J'ai fait des études en mathématiques pures et en physique. Ma passion m'a guidé vers une école d'ingénieur pour me permettre d'apprendre le C++, qui est le langage du secteur du jeu vidéo. Cela m'a permis, de combiner mes connaissances théoriques et pratiques afin de réaliser mes objectifs : concevoir des jeux vidéo. J'ai travaillé dans différents secteurs d'activité : aéronautique, divertissement, robotique, financier, scientifique, etc… en tant que programmeur et architecte logiciel. J'ai commencé par travailler sur le développement de système d'exploitation et de composants électroniques jusqu'à la réalisation de jeux et de logiciels plus haut niveau. J'ai utilisé de nombreux moteurs de jeux populaires et j'ai travaillé sur le développement de briques logicielles dans des moteurs du secteur. J'ai également travaillé sur les librairies standard C++ utilisées par certains moteurs. J'ai architecturé de nombreux moteurs qui géraient du rendu, de la physique et divers éléments d'IA pour le gameplay. Je travaille actuellement en tant qu'architecte dans une entreprise scientifique. J'enseigne le C++, l'architecture logicielle et le développement de jeu vidéo à des Master 2 depuis 2016. Je fais aussi des formations en entreprise sur le C++ depuis 2020.
J'écris des articles techniques sur le jeu vidéo pour certains magazines et je participe à des émissions pour vulgariser des points techniques. En 2023, j'ai organisé en collaboration avec un site de jeux vidéo un gros évènement lors du Xbox Showcase, où j'ai fait venir des grands noms du secteur.
Pour faire simple, le jeu vidéo est ma passion et c'est un plaisir de la partager !

Nom : LAVIGNE Quentin
Titre : Architecte logiciel
Email : quentin.lavigne@thermofisher.com

BIOGRAPHIE

Autodidacte en programmation, j'ai découvert celle-ci pratiquement en même temps que l'écriture puisque âgé de 6 ans et demi ! J'ai fait mes premières armes avec GW Basic, Turbo Basic puis QuickBasic avant de passer au Visual Basic vers mes 13 ans. Finalement, vers mes 14 ans j'ai franchi le pas d'apprendre le C++ afin de pouvoir programmer des jeux vidéo et suis tombé amoureux du langage...
D'un point de vue académique, j'ai fait des études de mathématiques pures (une autre de mes passions) durant lesquelles j'étais particulièrement attiré par la topologie et la géométrie différentielle. J'ai également un vif intérêt pour la physique et la cosmologie.
Professionnellement, j'ai eu l'occasion de programmer dans des domaines très variés : le management de projet, le calcul scientifique, la visualisation 3D temps réel, toujours en C++ !
Aujourd'hui, j'ai envie de faire découvrir mes passions

ROADMAP

DESCRIPTION

Les dates limites fixées doivent être rigoureusement respectées afin de garantir une évaluation équitable.

Jalon	Livrables attendus	Date limite	Moyens / formats
1	**Remise du projet**	19/12/2024 à 18h00	• Repository git avec la liste des membres du groupe envoyés sur le fil de rendu discord.
2	**Soutenance**	20/12/2024 de 13h00 à 19h00	• Soutenance orale

Gaming Campus - © Active Learning by Quest Education Group
Document Interne 2024-2025

EVALUATION

L'évaluation est conçue pour être holistique, prenant en compte non seulement le produit final, mais aussi le processus, les compétences acquises et les attitudes démontrées tout au long du projet. Les compétences sont classées en trois catégories principales : "savoirs" (connaissances théoriques), "savoir-faire" (compétences pratiques), et "savoir-être" (compétences interpersonnelles et attitudes).

SYSTÈME DE NOTATION

Savoirs (Connaissances)

- **Compréhension théorique** : Évaluée soit en amont du projet pendant la semaine théorie soit lors de la soutenance et restitution du projet. Cela permet de mesurer la compréhension des concepts fondamentaux et des connaissances liées au projet des étudiants.

Savoir-faire (Compétences)

- **Compétences techniques et application** : Évaluées à travers la soumission finale du projet. Cela inclut la qualité, la fonctionnalité, et la précision technique du travail produit.
- **Gestion de projet** : Évaluée en fonction de l'organisation, du respect des échéances, et de l'utilisation efficace des ressources. Cela peut être évalué à travers la documentation du projet et les journaux de processus.

Savoir-être (Attitudes/Compétences interpersonnelles)

- **Travail d'équipe et collaboration** : Évalués à travers des évaluations par les pairs et les membres du groupe. Les critères incluent la communication, la coopération, et la contribution aux tâches du groupe.
- **Autonomie et initiative** : Évaluées en fonction des contributions individuelles, de la capacité à travailler de manière autonome, et de la résolution proactive des problèmes.

Gaming Campus - © Active Learning by Quest Education Group
Document Interne 2024-2025

- **Soutenance** : Évaluées lors de la présentation finale du projet. Les critères incluent la clarté, la cohérence, et la capacité à articuler et défendre les résultats du projet.

GRILLE D'ÉVALUATION

Évaluation du produit fini	Note
Respect du cahier des charges et fonctionnalités implémentées	10
Interface utilisateur	5
Gestion des ressources	5
Total	**20**

Évaluation de la soutenance	Note
Aisance à l'oral	2
Justification des choix	4
Présentation de l'architecture	4
Respect du temps de parole	2
Organisation du travail	3
Améliorations proposées et problèmes rencontrés	3
Qualité des slides	2
Total	**20**

Questions/réponses	Note
Total	**20**

Notation sur le code	Note
Architecture	10
Algorithmique	10
Build system/git	10
Qualité du C++	10
Total	**40**

La note finale est la note sur 100 ramenée sur 20.

Gaming Campus - © Active Learning by Quest Education Group
Document Interne 2024-2025

Modalités de la soutenance :

- 50 minutes réparties de la manière suivante
    - 15 minutes de présentation PowerPoint ou similaire
    - 10 minutes de test du jeu
    - 20 minutes de questions
    - 5 minutes d’évaluation de la soutenance

EXAMEN INDIVIDUEL DE FIN DE PÉRIODE
Examen de 2h sur ordinateur comprenant :

- QCM
- Des exercices de C++